BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

”Material Minder”: A Voice-Enabled Mobile Application for Managing Construction Materials

Scientific Supervisor
lect. dr. Lazăr Ioan

Author
Văleanu Lucian-George

2025

ABSTRACT

Construction environments present unique challenges when it comes to managing materials, especially for older tradespeople who may struggle with traditional digital tools. Beginning in Chapter 2, the first theoretical chapter, this thesis explores accessibility-centered design principles, user experience models, and voice interface technologies, all of which inform the development of Material Minder, a mobile application tailored to this demographic. Through a comparative analysis of existing voice-enabled applications, the research identifies key design and functionality gaps that shaped the requirements for a more inclusive and efficient solution.

In Chapter 3, the second theoretical chapter, the focus shifts toward speech recognition technologies, outlining both their capabilities and limitations within high-noise, real-world contexts such as construction sites. This sets the foundation for the integration of a voice-first interaction model within the application.

Chapter 4 transitions into the practical development process, detailing the architectural structure, data logic, and interface design of *Material Minder*. The application combines a simplified UI with a natural speech-based material input system, aiming to reduce friction in everyday workflows. From project creation to cost estimation, each feature is built with clarity, speed, and accessibility in mind.

This thesis bridges theoretical research with applied mobile development, resulting in a tool designed not only to streamline material management but to empower an often-overlooked user group through thoughtful technology.

Contents

List of Figures

Chapter 1

Introduction

Working as a plumbing assistant for my father during my middle and high school years revealed a significant logistical difficulty: effective construction project material list management. Often working on multiple building sites, immediate access to paper and writing instruments was restricted, consequently forcing repeated visits to our vehicle or a quick search for accessible writing surfaces. Moreover, upon arrival at the supplier, the quickly scribbled notes sometimes proved tough to decipher, which could lead to possible mistakes and delays.

Although a smartphone gave a rapid note-taking option, it brought a novel set of issues, especially with reference to my father's grasp of modern technologies. Being a senior, his lack of experience with smartphone interfaces called attention to an usual issue older generations have adopting new digital tools. This finding showed a more general demand in the trades for more conveniently available technical solutions.

This study explains the building of a mobile application aimed to assist the production of material lists, hence addressing these pragmatic issues. Designed primarily to suit the demands of senior users and tradespeople who might have limited experience with smartphones, the program provides a user-friendly and straightforward design. The purpose is to offer a faultless and rapid approach to compile precise material lists, hence enhancing output and minimizing common job site challenges.

1.1 Tackling the problem

As the small example at the beginning of this introduction demonstrates, I feel to highlight that my direct experience revealed time is often a critical aspect within building site surroundings. The conventional techniques of material list compilation, which usually entail human transcription onto paper, continuously rose inefficiencies. These varied from the immediate disturbance of seeking for writing equipment on diverse building sites to the later difficulty of understanding hurriedly scribbled notes, often leading to potential blunders and project delays upon arriving at the supplier.

An intuitive, user-centered application gives a clear opportunity to greatly improve the logistical flow of a building project. By eliminating the necessity of physically writing down items, this digital approach can make the process of creating material lists considerably simpler and more accessible for every user, particularly addressing the needs of older tradespeople who may face a learning curve with modern technology.

The initial step in the creation of this application concentrated on building an intuitive and straightforward design. This basic idea ensures that users, regardless of their prior technological ability, may readily grasp and apply the application's capabilities. Building on this foundation, the second significant implementation was a Speech-to-Text (STT) functionality. This functionality simplifies list construction by allowing speech input, which saves the time and cognitive work generally connected with manual data entry. The clear display of these STT-generated lists then wraps in a readily decipherable and structured style. This direct technique avoids the issues of unreadable notes from the past, ensuring that when approaching the store, the list is fully clear, thereby improving the procurement process and enhancing on-site efficiency.

1.2 Thesis structure

The development of "Material Minder" required substantial research, focused on the design of a user-friendly interface and user experience specifically customized for tradesmen, with particular regard for senior users. This research also involved the implementation of a Speech-to-Text (STT) feature to considerably improve the list generating process.

Therefore, the succeeding chapters of this thesis are structured as follows:

Chapter 2 forms the main theoretical foundation of this study. It will comprehensively cover core User Experience (UX) and User Interface (UI) principles, with a specific focus on design concerns for accessibility, simplicity, and usefulness among

senior users and tradesmen. Furthermore, this chapter will go into the underlying ideas and practical features of Speech-to-Text (STT) technology, which is central to the application's functionality.

Chapter 3 will provide a comparative review of existing applications using Speech-to-Text (STT) functionality for note or list creation. Given that "Material Minder" is targeted for a narrow user group, directly equivalent apps were not discovered. Consequently, this chapter will study more general-purpose VTT-enabled platforms to find common traits, strengths, and limits relevant to a construction setting, ultimately driving the design for "Material Minder."

Finally, Chapter 4 will discuss the concept and implementation of "Material Minder." This chapter will address the methodological approach, specific design choices made based on the theoretical framework, and the technical execution of the application, including the integration of its core features.

1.3 Declaration of Generative AI and AI-assisted technologies in the writing process

I affirm that this thesis is the direct result of my personal effort and research. I have not knowingly solicited or obtained any unauthorized aid in its completion. Google Gemini 2.5 Flash was utilized as a tool to aid the structuring of chapters, produce initial ideas, provide instructive examples, and assist in grasping complex concepts discovered throughout my research. Additionally, QuillBot Premium was utilized for paraphrasing selected sentences to enhance academic tone, improve clarity, and ensure linguistic precision within the thesis. All information, analysis, and conclusions contained in this thesis remain my own original work, and any AI-generated or AI-assisted output has been thoroughly scrutinized, validated, and incorporated under my full intellectual control and responsibility.

Chapter 2

Theoretical Foundation and Design Principles

This chapter lays the foundation for the creation and development of Material Minder by discussing fundamental theoretical concepts and established principles. The text commences with a broad examination of User Experience (UX) and User Interface (UI) design, gradually refining its focus to cater to the particular requirements of senior users and tradesmen. Subsequently, the chapter digs into Speech-to-Text (STT) technology, outlining its mechanisms, advantages, problems.

Understanding the digital landscape, particularly within the intended audience, is crucial for effective application design. While the overall digital landscape in Romania demonstrates near-universal mobile phone ownership, with 95.7% of the total population owning a mobile device, and a striking 96.2% of internet users aged 16+ owning a smartphone [SK25], understanding the specific digital engagement of older demographics requires closer examination. Elderly individuals sometimes describe feeling overwhelmed by the quick rate of technology innovation and widespread digital tool usage [Dob22].

A specific study concentrating on internet usage among senior adults in the Bucharest-Ilfov area indicated that 44.3% of their survey respondents actively use the internet [Dob22]. This regional data, while not national, provides crucial information into internet penetration among the elderly population in Romania. Considering the DataReportal conclusion that nearly all internet users own a smartphone, it can be deduced that a significant number of Romanian seniors online are also utilizing smartphones.

The same study emphasizes important facilitators of internet access for seniors, particularly emphasizing family/grandchildren (89.5%) and friends/neighbors (69%) as major influences [Dob22]. This shows that social support plays a vital role in bridging the digital divide for older persons, making technology adoption often a community effort. Furthermore, their key online activities largely revolve around

social contact (online chats, social media) and information intake (news), with online buying being a less important activity [Dob22]. Comprehending their digital habits and dependence on social networks for adoption is vital for designing applications such as "Material Minder," which strives to expedite practical activities and integrate smoothly into their digital existence.

2.1 Principles of User Experience (UX) and User Interface (UI) Design

2.1.1 User Experience, Human-Computer Interaction, and Garrett's Model

Understanding the essence of User Experience (UX) and Human-Computer Interaction (HCI) is fundamental to designing effective digital products. Traditionally, HCI primarily focused on aspects like a product's functionality and its usability - essentially, whether it does what it's supposed to do and how easy it is to use. However, the concept of UX has evolved significantly beyond these pragmatic qualities. Modern UX encompasses a much broader spectrum, including the emotional and sensory aspects of interaction, often referred to as "hedonic quality"[JG07]. This means considering factors like attractiveness, fun, and even how a product conveys a brand's proposition, all of which contribute to a holistic positive user experience.

Jetter and Gerken's simplified model of User Experience clarifies this expanded view of HCI and UX. Their model posits that UX is the overarching outcome derived from the intricate relationship between the user, the product, and the organization behind it. It highlights that both user values (what the user seeks and feels) and organizational values (business goals, branding) significantly influence this experience. The product, therefore, acts as a crucial interface, mediating these values. The user's interaction with the product is perceived on two levels: a pragmatic level, encompassing functionality and usability, and a hedonic level, which includes attributes like stimulation, identification, and evocation. Similarly, the organization's interaction with the product involves everyday operations (related to functionality and usability) and broader marketing, branding, and business communication strategies. All these elements, in their various forms, collectively contribute to the overall User Experience.

While Jetter and Gerken provide a conceptual model of UX, Jesse James Garrett's widely recognized "Elements of User Experience" framework offers a more practical, layered approach to understanding and building UX. This model decomposes UX into five distinct planes, moving from the abstract to the concrete:

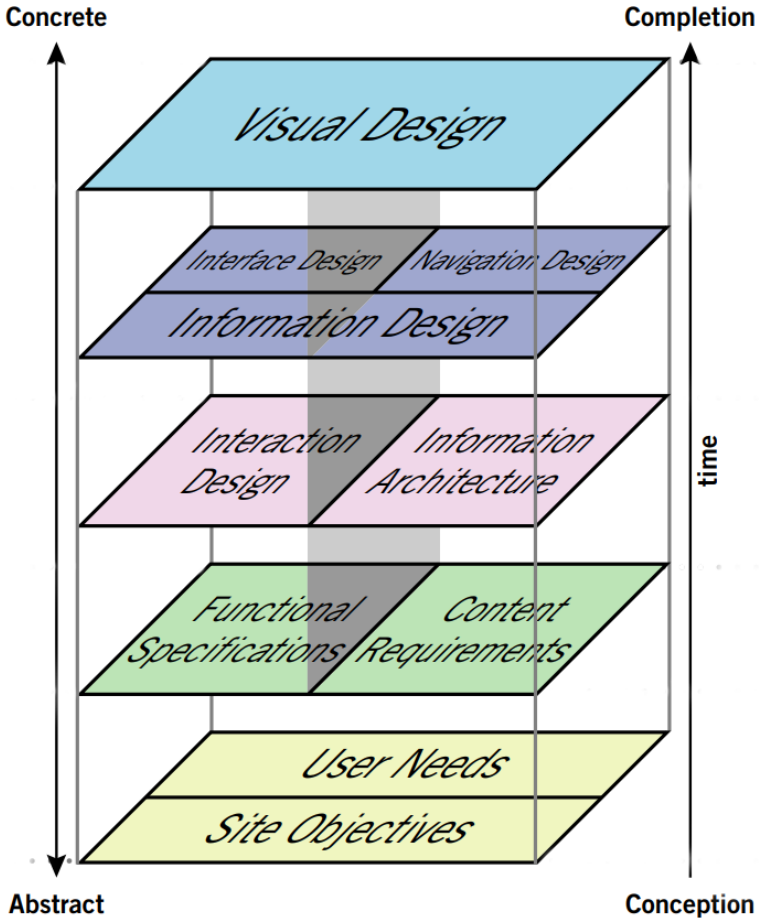

Figure 2.1: The Five Planes of User Experience, as conceptualized by Jesse James Garrett.[Gar03]

Strategy Plane: This foundational layer defines the user needs and the site objectives. It's about what the user wants to get out of the product and what the business wants to get out of the product.

Scope Plane: Based on the strategy, this layer determines the functional requirements and content requirements of the product. It defines what features and information the product will include.

Structure Plane: This layer defines the interaction design (how users will interact with the system) and information architecture (how content is organized).

Skeleton Plane: This layer focuses on the interface design (placement of buttons, forms, etc.), navigation design (how users move through the site), and information design (how information is presented).

Surface Plane: This is the most concrete layer, representing the visual design - what the user actually sees. It includes aesthetics, layout, and sensory details that make up the final user experience.

Both Jetter and Gerken's conceptual model and Garrett's layered framework provide valuable lenses through which to understand and design for User Experience,

ensuring that both the underlying principles and the practical implementation are considered.

2.1.2 User Interface and Software Design

Based on Nacheva's paper, when discussing User Interface design, we refer to the front-end of an application, which can be both software or hardware [N+15]. Simply put, the user interface's design can be seen as a representation of "how the software communicates within itself, with systems that interoperate with it, and with humans who use it."[N+15] An interface suggests a particular kind of behavior as well as a flow of information (such as data and/or control).

Building upon this, effective User Interface and software design extend beyond mere technical communication. As the field of User Experience emphasizes, the design process for software must consider both the practical aspects (functionality and usability) and the more subtle, emotional connections a user forms with a product. This means that while the interface needs to be functional and usable for everyday operations, allowing users to efficiently complete tasks, it also needs to incorporate "hedonic qualities". These hedonic qualities in UI design can manifest as an attractive visual design, engaging interactions, or elements that evoke positive feelings, making the software not just efficient but also enjoyable and memorable to use. Furthermore, software and UI design are significantly influenced by the organization's values, such as branding and marketing goals, which aim to deliver a consistent brand experience through all product touchpoints. Therefore, modern software design, particularly in its user interface, strives to create an "elegant equilibrium" that delivers value for both the user and the organization.

2.2 Designing for Specific Demographics: Elderly Users

As global demographics shift towards an ageing population, the importance of designing technology that is accessible and intuitive for elderly users becomes increasingly critical. While smart devices offer numerous benefits, older individuals often face challenges in their adoption and effective use due to factors such as reduced familiarity with new technologies, along with potential cognitive, physical, and sensory changes associated with ageing. These challenges highlight a significant gap in current research and design practices, emphasizing the need for tailored user interface (UI) and user experience (UX) solutions.

Designing effectively for elderly users necessitates a keen awareness of their unique needs, moving beyond standard design conventions that might inadvertently create barriers. A primary concern is readability and visual clarity. Research

suggests that font sizes should be significantly larger on mobile applications, ideally ranging from 36 to 48 points, utilizing sans-serif styles (such as Roboto for Android) and clear, simple language [AC20]. Furthermore, the ability for users to adjust text size is crucial for personalization [AC20]. High contrast between text and background colors is also vital for legibility, with customization options preferred to accommodate varying visual preferences [AC20]. Beyond text, icons should be straightforward and unambiguous, ideally accompanied by text labels to prevent misinterpretation, particularly avoiding complex or multi-meaning symbols like the ”hamburger” icon, which can disorient older users and hinder their performance [AC20]. Studies specifically evaluating icon design for seniors indicate that larger icon sizes (e.g., 72 to 96 pixels) are more discernible than smaller ones (48 pixels), and caricature-style icons tend to be the most recognizable [AC20].

Navigational simplicity and consistent assistance are equally important design considerations. User interfaces for elderly individuals should minimize complexity, avoiding slide-out menus and prioritizing a clear ”back” button to the main menu at all times [AC20]. The use of pop-ups and multiple, convoluted actions should also be limited to streamline interactions [AC20]. To foster a sense of security and support, a readily accessible help button ought to be consistently present within every activity window, offering immediate guidance when needed [AC20]. Specific regional studies, such as those focusing on Thai seniors, further refine these guidelines, suggesting optimal font sizes (12 or 16 points) to ensure content fits on a single screen, reducing the need for scrolling, which many elderly users find challenging [AC20]. Additionally, a green-toned screen with 75% brightness, utilizing Arial Unicode MS font, has been identified as a particularly suitable configuration for this demographic [AC20].

To truly empower older adults in the digital space, the next frontier in UI/UX design involves personalized and adaptive systems. An intelligent UI/UX system can be optimized for senior citizens by dynamically adjusting to their individual cognitive and sensory functionalities, as well as the characteristics of the digital content they are consuming [YLL16]. Such systems rely on a ”user model” that captures both static information (like age and education) and dynamic aspects (such as current cognitive function, emotional state, and health status) to tailor the experience [YLL16]. Concurrently, a ”contents model” can describe elements of the digital material, from basic attributes to usability-related features like subtitle appearance, screen brightness, audio volume, and even the size of interactive buttons [YLL16]. By leveraging these models, the UI/UX system can intelligently adapt components such as font type and size, or the amount of information displayed, to perfectly match the user’s current needs and the device’s screen size [YLL16]. This adaptive approach holds significant promise for enhancing smart device usability, boosting

digital content consumption, and ultimately improving the overall quality of life and social perception for senior citizens [YLL16].

It is important to note that while extensive research supports specific design considerations for elderly users, the provided papers focus exclusively on this demographic. Therefore, this discussion does not include specific design recommendations for "tradespeople," as that information was not present in the analyzed sources.

Key Design Guidelines for Accessible Mobile Interfaces

The proliferation of mobile devices and applications has fundamentally reshaped daily life, presenting both immense opportunities and significant design challenges. As mobile application development continues its rapid growth, developers frequently grapple with the complexities of meeting diverse user expectations, a challenge largely dictated by the application's User Interface (UI) and User Experience (UX) [SBT+22]. Effective design must consider a wide array of parameters, moving beyond isolated issues to embrace a holistic view that ensures inclusivity and usability for all.

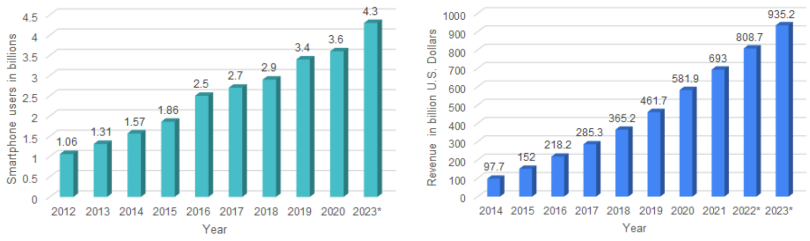

Figure 2.2: (a) available count of smartphone users worldwide and (b) global mobile application market size[SBT+22]

Despite the pervasive use of smartphones and the integrated assistive technologies they offer (such as VoiceOver for iOS and TalkBack for Android) [YR19], a substantial portion of mobile applications still present significant accessibility barriers. A study examining 479 Android applications revealed that a remarkably high percentage of apps contain accessibility issues, with 94.8% exhibiting violations, 97.5% having potential violations, and 66.4% displaying warnings [YR19].

These issues predominantly stem from critical design oversights, including a lack of proper element focus, missing element descriptions, insufficient text color contrast, inadequate spacing between interactive elements, and text fonts or elements that are smaller than recommended minimum sizes [YR19]. Common graphical user interface (GUI) elements like TextView, ImageView, View, Button, and ImageButton are frequently implicated in these accessibility shortcomings, highlight-

ing that even fundamental components can contribute to exclusionary experiences if not meticulously designed [YR19].

Addressing accessibility is particularly pressing for older individuals, who often experience age-related disabilities that can make interacting with mobile interfaces exceptionally difficult [DBM14]. Generic design guidelines are often insufficient for this demographic, necessitating a more focused approach. Key guidelines for enhancing accessibility in mobile interfaces for older people emphasize several critical areas:

- **Interaction Design:** Prioritize touch interaction over complex gestures. Ensure touch target sizes are generous enough to accommodate varying motor skills and reduce the likelihood of accidental presses [DBM14].
- **Visual Design:** Implement larger font sizes and maintain high contrast ratios between text and background elements to improve readability. Designs should avoid visual clutter, presenting information clearly and concisely. Icons should be easily discernible and intuitively understood, potentially benefiting from accompanying text labels [DBM14].
- **Navigation Simplicity:** Navigation paths must be intuitive, consistent, and straightforward. Overly complex menu structures or hidden navigation elements should be avoided to prevent confusion and disorientation [DBM14].
- **Feedback Mechanisms:** Provide immediate and unambiguous feedback for all user actions. This confirms that an input has been registered and helps users understand the system's state, reducing frustration [DBM14].
- **Content Clarity:** Use clear, concise, and easy-to-understand language, avoiding jargon or overly technical terms that might impede comprehension [DBM14].

Beyond traditional touch and visual interfaces, Mobile Voice User Interfaces (M-VUIs) represent a crucial advancement in accessibility, especially for users with limited hand dexterity caused by various motor impairments [CW16]. M-VUIs enable entirely hands-free, voice-only interaction, significantly enhancing independence for these individuals by allowing them to control mobile devices through voice commands alone [CW16]. However, integrating voice interactions effectively presents its own set of user experience challenges, particularly concerning the difficulty users often encounter in learning and discovering available voice commands [CW16]. Therefore, ongoing research is essential to develop methods that improve the visibility and learnability of voice commands, ensuring that the power of M-VUIs can be fully leveraged by those who stand to benefit from them the most [CW16].

2.3 Speech-to-Text (STT) Technology

Building upon the established aspects for an optimal user experience, the proposed solution for enhancing the workflow of ageing tradespersons, particularly within the ”Material Minder” application, integrates a Speech-to-Text (STT) feature.

Speech-to-Text (STT) technology stands as a fundamental component in modern communication applications, facilitating the conversion of spoken language into written text [TPS+18]. This process, often rooted in articulatory and acoustic-based speech recognition, enables seamless information conveyance across networks, acting as a vital mediator in transforming speech signals into a textual format for various digital platforms [TPS+18].

Advantages of STT for List Creation

The adoption of Speech-to-Text (STT) technology offers significant advantages for list creation, particularly in scenarios where traditional manual data entry methods are impractical or challenging. A primary benefit of voice-based interfaces is their hands-free operation, which is invaluable in environments where users' hands may be occupied or dirty, such as during construction or maintenance tasks [HGK18]. This capability allows for seamless data input without interrupting ongoing physical work, thereby enhancing efficiency and safety.

Furthermore, STT systems provide a crucial tool for improving accessibility and ease of use, especially for individuals who may be less familiar with modern digital interfaces or experience difficulties adapting to them, a common challenge observed in traditional trades and small businesses [TAHPC24]. By enabling users to control databases and manage inventory through natural voice commands, STT platforms function as an ”easy to control” virtual assistant, significantly lowering the barrier to entry for digital management systems [TAHPC24]. This voice-based interaction facilitates initial engagement with new technologies for individuals who might otherwise be hesitant or unable to utilize them effectively, streamlining critical processes such as inventory updates and material tracking within applications like ”Material Minder” [TAHPC24]. The capacity to efficiently ”query and fetch data” via speech further enhances the overall utility and responsiveness of such systems for dynamic list management [HGK18].

Moreover, specific applications of Speech-to-Text (STT) technology, such as voice-picking systems in warehousing, underscore its significant advantages for list creation and management [Dat25]. These systems offer a substantial improvement over traditional paper-based methods, directly contributing to increased efficiency, accuracy, and overall throughput in operations [Dat25]. A pivotal benefit is the hands-free capability, which allows workers to concentrate fully on their tasks, thereby

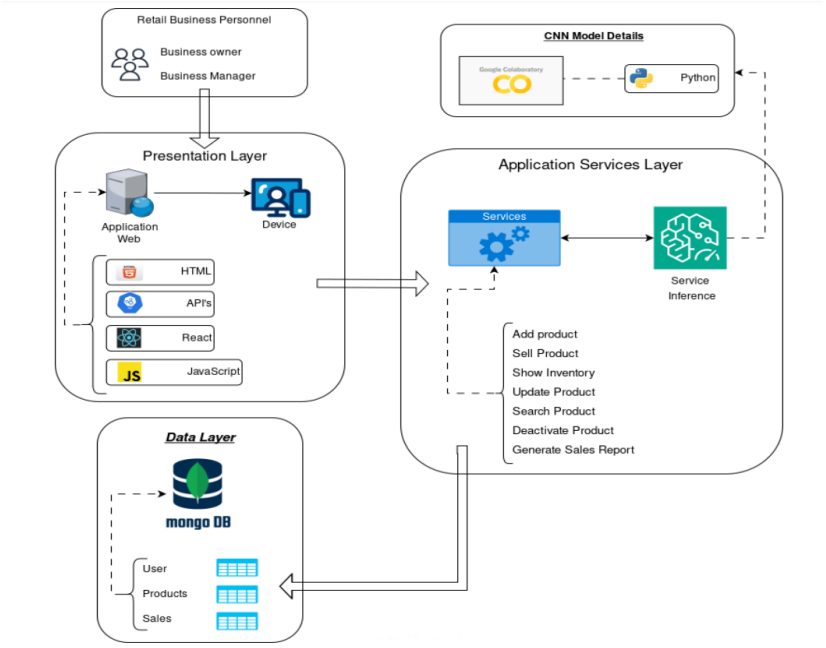

Figure 2.3: Logic Architecture Diagram of a solution for a small business[TAHPC24]

reducing picking errors by approximately 67% compared to manual pen-and-paper processes [Dat25]. This feature is particularly advantageous in challenging environments, such as cold storage, where workers can maintain proper protective gear without needing to remove gloves for data entry [Dat25].

Voice-directed systems also enable greater inventory control by facilitating real-time updates and feedback. Workers can vocally report issues like incorrect locations, inventory shortages, or product damage, leading to automatic system updates [Dat25]. This real-time interaction enhances communication and decision-making. Furthermore, modern voice-picking solutions incorporate highly advanced speech recognition software capable of accurately interpreting spoken words, even amidst ambient noise and various accents. Many systems also offer multi-language support, allowing workers to operate in their most comfortable language, which not only boosts productivity and safety but also expands the available labor pool [Dat25]. These features collectively demonstrate how STT can transform list-centric tasks into more efficient, accurate, and user-friendly processes, extending its utility to diverse workforces and operational contexts.

Challenges and Limitations of STT

Despite the significant advancements in Speech-to-Text (STT) technology and its growing integration into various human-computer interaction (HCI) systems, several inherent challenges and limitations persist, impacting its accuracy and widespread applicability [BAJ+23]. These limitations stem from a complex interplay of environ-

mental factors, speaker-specific characteristics, and the intricacies of speech signal processing.

One of the most prominent challenges for STT systems is background noise [BAJ+23]. Environmental sounds, ranging from moderate ambient noise to sudden loud disruptions, can severely degrade the quality of the speech signal, making it difficult for the system to accurately differentiate between speech and noise. This often leads to misinterpretations or a complete failure to transcribe.

Speaker-related variability also presents substantial hurdles. Accents and dialects introduce considerable diversity in pronunciation, rhythm, and intonation, which can significantly reduce recognition accuracy for models not specifically trained on such variations [BAJ+23]. Similarly, the speed of speech can pose a challenge; rapid speech may result in dropped words or phonetic reductions, while unusually slow speech might alter typical phoneme durations, affecting recognition algorithms [BAJ+23].

Furthermore, STT systems must contend with speaker variability, encompassing differences in vocal pitch, timbre, and articulation unique to each individual. This necessitates robust models capable of generalizing across a broad spectrum of voices [BAJ+23]. The distance of the speaker from the microphone also plays a critical role, as increased distance typically leads to lower signal-to-noise ratios and reduced clarity [BAJ+23]. Lastly, emotion conveyed through speech can alter vocal characteristics, impacting the acoustic patterns that STT systems rely upon for accurate transcription [BAJ+23]. Addressing these multifaceted challenges is crucial for developing STT systems that are reliable and effective across diverse real-world conditions.

Chapter 3

Comparative Analysis

During the initial research phase for Material Minder, it became apparent that no existing application directly mirrored its specific purpose: simplifying material list creation for tradespeople, especially the elderly, through an intuitive interface and integrated Speech-to-Text (STT) functionality. This absence required a broader analytical approach. This chapter, therefore, provides a detailed analysis of 'Voicenotes', an application integrating robust Speech-to-Text (STT) capabilities for note generation.

3.1 Application Profile: Voicenotes

3.1.1 Introduction to Voicenotes

Voicenotes is an AI-powered note-taking application designed to redefine how individuals and teams capture, organize, and leverage spoken information. It seamlessly integrates voice recording with advanced artificial intelligence to convert spoken input into actionable, searchable, and easily repurposable text. The application's core objective is to streamline the note-taking process, augment human memory, and facilitate content creation by harnessing state-of-the-art AI technologies.

The application positions itself as an "intelligent note-taking app" and an "AI-enhanced 'second brain'". This positioning directly addresses a prevalent human challenge: the inherent limitations of memory and the substantial cognitive burden associated with recalling and organizing information. In an increasingly information-dense environment, tools that promise to extend human cognitive capacity and provide reliable information storage hold significant appeal across a diverse spectrum of users, ranging from busy professionals to individuals managing intricate personal lives. This differentiates Voicenotes from mere transcription tools by offering a comprehensive solution for cognitive offloading and information management. The application effectively tackles the common problem of capturing fleeting ideas

and detailed information without the cumbersome process of manual typing, subsequently ensuring that this information remains easily accessible and useful long after its initial capture.

Functionally, Voicenotes provides robust AI transcription capabilities, an AI-powered question-and-answer system that operates over previously recorded notes, and automated content repurposing features, including the generation of summaries, to-do lists, and various content drafts. It supports transcription in over 100 languages and asserts a high level of accuracy. Voicenotes targets a broad audience, including entrepreneurs, writers, students, and professionals in sectors such as sales, customer support, and recruiting, as well as individuals for personal use cases like voice journaling and managing home projects.

3.1.2 Founders' Journey and Vision

Voicenotes was founded by Jijo Sunny and Aleesha in 2023. Jijo Sunny, also co-founder and CEO of "Buy Me a Coffee", contributes over a decade of experience as a Senior Software Engineer and Technology Lead, specializing in scalable back-end systems and innovation across various programming languages and cloud platforms. Aleesha serves as the main developer of the application[Yah24].

The conceptualization of Voicenotes stemmed from a deeply personal experience of the founders. Following a challenging period, their need for a reliable method to record and recall critical medical information from doctors and nurses highlighted a profound requirement for an accurate transcription tool [Yah24]. This personal necessity directly influenced the product's design, prioritizing accuracy, reliability, and data security. This foundation fosters a unique connection with its users, building trust, especially concerning privacy, which is a key differentiator in the AI landscape.

The founders' vision for Voicenotes is characterized by a commitment to user privacy, data longevity, and design simplicity. They assert that all user data is encrypted at rest, is explicitly never used for AI training, and is accessible only via authenticated user requests. Voicenotes adheres to GDPR compliance, with SOC 2 certification actively in progress. This strong stance on privacy addresses a significant concern in AI-powered applications, potentially attracting users handling sensitive information [Yah24].

Furthermore, the emphasis on "simplicity and beauty in design" is a core tenet, articulated by Jijo Sunny as a significant investment of time and effort to ensure an an intuitive user experience. This dedication to design has been consistently noted positively by users.

Strategically, Voicenotes is entirely self-funded, a decision made to maintain full

autonomy over its vision and product direction, prioritizing long-term values without external influence. While this bootstrapping approach ensures control, it may impact the pace of scaling and market penetration compared to heavily funded competitors, necessitating reliance on organic growth and superior product experience.

Jijo Sunny identifies the application's strategic differentiation through its elegant design, utilization of advanced AI models, and the unique "Ask My AI" feature. The long-term vision includes expanding cross-platform functionality to serve as a "real-time assistant" and integrating features such as automatic conversion of voice notes into to-do lists with reminders.

3.1.3 Core Product Features and Functionality

Voicenotes is characterized by a robust suite of features that extend beyond basic voice recording, leveraging artificial intelligence to establish a comprehensive productivity tool.

- **Voice Notes Capture:** This fundamental capability enables users to record thoughts, ideas, lists, and memories on compatible devices. This frictionless capture mechanism aims to address the challenges of traditional note-taking and capturing fleeting insights.
- **Smart Transcription:** Utilizing state-of-the-art AI, Voicenotes provides highly accurate transcriptions, reportedly surpassing some established competitors. It supports over 100 languages and demonstrates proficiency in transcribing whispers and recognizing names, with transcriptions generated instantly.
- **AI-Powered Retrieval ("Ask AI"):** This feature allows users to query their past notes to receive instant, contextually relevant answers. Positioned as a "personal AI assistant," it leverages advanced models, such as Chat GPT 4 Turbo [Yah24], to process and transform information, thus addressing the issue of scattered notes and providing actionable intelligence.
- **Content Repurposing ("Create"):** Voicenotes automatically transforms recorded notes into various formats, including concise summaries, actionable to-do lists, blog post drafts, and emails, customizable via user-defined prompts. This functionality aims to enhance productivity by streamlining content generation.
- **Meeting Recording & Management:** The application supports recording of both in-person and online meetings without requiring a bot, offering instant summaries and full transcripts. It is compatible with major meeting platforms

and facilitates sharing of notes. Recent enhancements include speaker diarization and timestamps for structured transcripts.

- **Privacy and Security:** A critical aspect of Voicenotes' design is its explicit commitment to privacy and security. All user data is encrypted at rest, is never utilized for AI training, and is accessible only through authenticated user requests. The application adheres to GDPR compliance, with SOC 2 certification in progress. This stance aims to build trust, particularly for users handling sensitive information.
- **Cross-Platform Availability & Sync:** Voicenotes ensures broad accessibility across web, iOS, Android, macOS, watchOS, WearOS, and as a Chrome extension. Seamless multi-device synchronization ensures notes are consistently available across all user platforms.

3.2 Synthesis of Comparative Findings

In summary, while Voicenotes stands as a highly sophisticated general-purpose AI-powered note-taking application, excelling in broad transcription accuracy, intelligent retrieval, and versatile content repurposing across numerous platforms, Material Minder carves out its unique and critical niche through specialized functionality. Voicenotes' strength lies in its comprehensive utility for diverse users, transforming spoken information into actionable insights for personal and professional use. Conversely, Material Minder focuses its innovation on a specific, underserved demographic and use case: simplifying material management for tradespeople, especially elderly users, within the demanding environment of construction sites. Unlike Voicenotes, which offers a wide array of features for generic note organization, Material Minder's design is meticulously tailored to the practical workflow of construction, prioritizing extreme ease of use, hands-free operation in noisy environments, and dedicated functionality for creating, tracking, and estimating construction material lists. This comparative analysis thus underscores Material Minder's distinct value proposition, highlighting its role as a purpose-built solution that fills a specific gap in the market by prioritizing accessibility and efficiency for a particular industry and user group.

Chapter 4

Practical Development of ”Material Minder”

This chapter transitions from the theoretical foundations discussed previously to the practical design and implementation of ”Material Minder.” It details how the application addresses the identified challenges in construction material management, particularly for senior users, by leveraging principles of user-centric design, robust user interfaces, and advanced Speech-to-Text (STT) technology. This section will first provide an overview of the application's core philosophy and then delineate its functional and non-functional requirements, demonstrating their alignment with the theoretical framework.

4.1 Core Design Philosophy

The development of Material Minder is guided by several key design philosophies, directly stemming from the theoretical considerations outlined in Chapter 2:

4.1.1 User-Centric Accessibility

Material Minder prioritizes the user experience (UX) and user interface (UI) to ensure maximum accessibility and ease of use, particularly for elderly individuals and those with motor or visual impairments. This involves principles such as large, legible font sizes (minimum 16pt for primary text), high contrast ratios, clear and unambiguous icons, simplified navigation, and minimal use of complex gestures. The design fosters trust and reduces digital apprehension.

4.1.2 Voice-First Efficiency

The application integrates a voice-enabled input system as a primary method for material entry, allowing for hands-free interaction. It provides immediate feedback on transcribed speech and intelligently matches input to a comprehensive material catalogue.

4.1.3 Streamlined Material Management

Material Minder offers robust functionalities for managing projects and materials. It includes a global material catalogue with user expansion and automatic cost estimation, aiming to improve accuracy and real-time expense insights.

4.1.4 Robustness and Reliability

The application emphasizes performance, reliability, and data integrity, ensuring a responsive experience with secure and consistently available user data.

4.2 Material Minder Requirements

The Material Minder application is designed to fulfil a comprehensive set of core functionalities, engineered to provide an intuitive and efficient experience for its target users. These functionalities are systematically elaborated in the following subsections, detailing the specific capabilities and interactions that the application offers. A high-level overview of the system's interactions with its users is presented in Figure 4.1.

4.2.1 Functional requirements

- **Initial Access Provisioning:** Upon successful installation and initial launch on the user's device, the application shall automatically provision user access, eliminating the need for explicit manual permission grants for fundamental operation.
- **Project Management Functionality:** The system shall provide comprehensive capabilities for the user to manage individual projects, including functionalities for creation, viewing, modification, and deletion.
- **Construction Item Catalog Access:** The application shall present the user with a complete and browsable catalog of pre-defined construction items.

- **Individual Item Management:** The system shall enable the user to manage individual construction items within the catalog, encompassing viewing, adding new entries, updating existing details, and deleting items.
- **Project Item Association and Quantity Management:** The system shall support the association of construction items from the catalog with specific user projects. Additionally, it shall facilitate the updating of quantities for these associated project items.
- **Voice Input Permission Handling:** Upon initiation of voice-based input for adding construction items to a project, the system shall prompt the user for audio recording permission via a dedicated modal interface.
- **Voice-Enabled Item Addition and Display:** Subject to successful acquisition of audio recording permission, the system shall enable the user to add construction items to their project via voice input. Newly added items shall be dynamically displayed at the forefront of the project's item list immediately following the completion of voice recording.
- **Project Detail View and Estimated Cost Display:** When viewing the detailed project screen, the system shall present a simplified list of associated construction items, displaying each item's name and its corresponding quantity. Furthermore, an estimated total price for the project's materials shall be displayed at the bottom of this screen. It is important to note that this price is considered an estimation, as the integrated construction item catalog originates from a singular major provider, and its pricing may vary from local supplier rates.

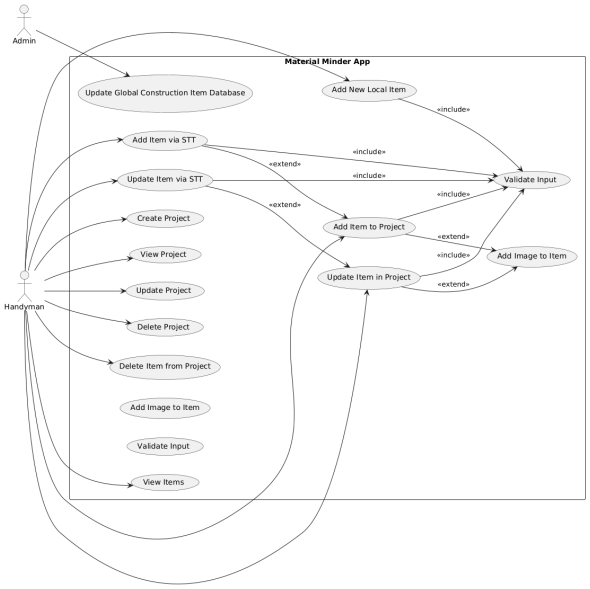

Figure 4.1: Use Case Diagram for the Material Minder Application

4.2.2 Non-functional requirements

- **Performance:**
  - **Response Time:** The system shall respond to user interactions (e.g., project creation, item search, voice command processing) within an average of 2 seconds, and never exceeding 5 seconds under normal operating conditions.
  - **Loading Time:** All major application interfaces (e.g. project list, construction items catalogue) shall load completely within 3 seconds on a typical mobile device with a stable network connection.
- **Usability:**
  - **Learnability:** First-time users must be able to successfully create a project and add items via voice input within 5 minutes of initial application launch without external assistance.
  - **User Interface Consistency:** The application must maintain a consistent and intuitive user interface design across all features and platforms.
  - **Error Handling:** The system must provide clear, actionable feedback for all user errors and system failures.
- **Security:**

  - **Authorization:** The system must ensure that users can only access, modify, or delete their own projects and associated data.
  - **Audio Privacy:** Recorded voice data must be processed solely for the purpose of transcription and item identification, and explicitly not used for any external AI model training or retained beyond necessary processing.

- **Reliability:**
  - **Availability:** The application must maintain an uptime of at least 99.5% (excluding scheduled maintenance periods).
  - **Data Integrity:** The system must ensure that all project and item data remains consistent and uncorrupted, even in the event of unexpected application termination or device failure.
  - **Fault Tolerance:** The application must gracefully handle and recover from common errors such as network interruptions or invalid user inputs without data loss.

- **Maintainability:**
  - **Modularity:** The system architecture must be modular, allowing for independent updates and enhancements of features without affecting core functionalities.
  - **Documentation:** All code and design decisions must be comprehensively documented to facilitate future development and maintenance.

- **Portability/Compatibility:**
  - **Platform Compatibility:** The application must be compatible with major mobile Android operating systems
  - **Device Compatibility:** The application shall function correctly across a range of screen sizes and resolutions typical of modern smartphones and tablets.

4.3 System Architecture and Design

The system architecture of Material Minder was intentionally designed to balance performance, maintainability, and user-centered responsiveness. Given the application's core focus on accessibility and real-time interaction, particularly through voice input, the architecture follows a layered approach that cleanly separates concerns

across presentation, business logic, and data layers. This separation not only simplifies development and testing, but also ensures long-term scalability and modularity as the application evolves. Furthermore, the system is built with platform-native technologies and modern architectural standards to support offline availability, fast UI response times, and secure data handling. The following sections describe the core components of the architecture in detail.

4.3.1 Frontend – User Interface (UI) and User Experience (UX)

In designing the user interface for Material Minder, I focused on creating an experience that feels natural, predictable, and accessible, especially for older tradespeople who may have limited familiarity with digital tools. My main priority was clarity: every screen, button, and interaction had to be immediately understandable without requiring prior technical experience.

To achieve this, I designed the layout to be clean and visually calm. I avoided overwhelming the user with too many elements at once, and instead structured the screens around a single clear action or purpose. I used large, readable text, obvious touch targets, and consistent spacing to ensure that users could navigate the app without hesitation.

Throughout the design process, I frequently referred back to the key UX principles I identified in the theoretical chapter: simplicity, feedback, predictability, and user control. These guided every decision I made, from button placement to color contrast. My goal was not just to make the app usable, but to make it feel comfortable, even for someone who had never used a smartphone app before.

To validate my design direction before implementation, I created a low-fidelity prototype using Figma. This allowed me to visualize the app's structure, layout, and navigation flow in a way that aligned with the user experience principles I intended to follow. The prototype served as a blueprint during development and helped identify early usability improvements, such as simplifying screen transitions and adjusting button placements for better touch ergonomics.

Figure 4.2 illustrates an excerpt from the initial Figma prototype, showcasing the main screens for project creation and material management.

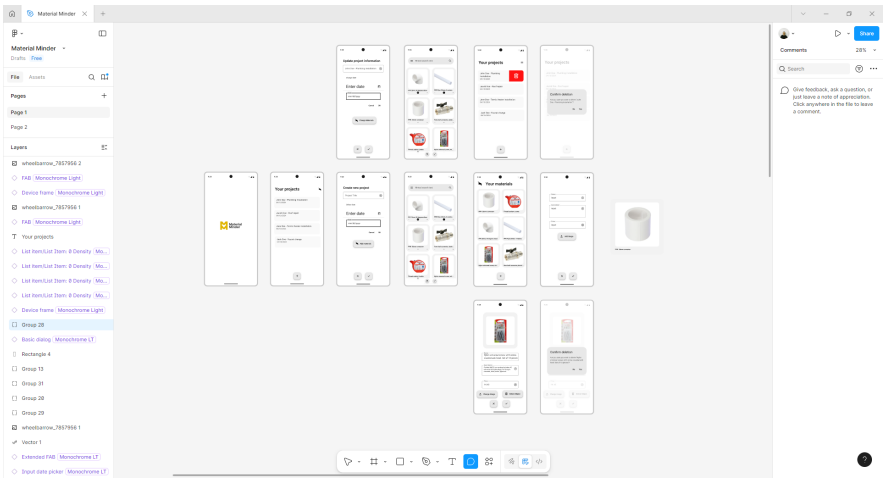

Figure 4.2: Excerpt from the Figma prototype showing the UI layout.

4.3.2 Backend – Responsibilities and Logic

When designing the backend for Material Minder, my main objective was to ensure that the application would be able to manage user data efficiently, securely, and reliably. I structured the backend around clear responsibilities: storing construction materials, managing user projects, calculating material costs, and supporting consistent data access between sessions.

To achieve this, I first mapped out the core entities in the system and how they relate to each other. At the heart of the backend is the concept of a "project," which groups together a list of construction materials. Each material entry within a project includes additional attributes like quantity, optional cost, and can either come from the global catalogue or be user-defined.

The backend handles all the data operations needed to make the mobile app functional: creating and editing projects, modifying material lists, retrieving saved data, and deleting entries safely. I made sure that each of these actions follows a well-defined logical flow, with proper data validation to prevent errors or inconsistencies.

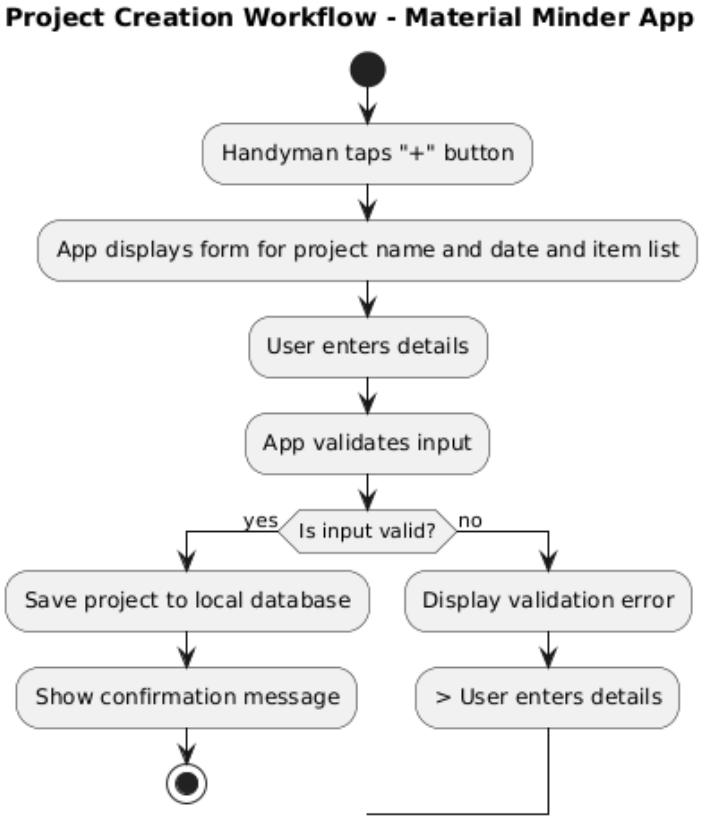

Figure 4.3: Project Creation Workflow Diagram.

To simplify the structure and improve maintainability, I organized the logic into distinct layers: one responsible for processing requests and handling validation, and another for interfacing with persistent storage. This separation allowed me to keep the codebase clean and adaptable to future updates or new features.

One specific challenge I addressed was ensuring data integrity when materials are deleted, both from individual projects and from the global catalogue. I introduced checks and confirmation steps in the logic to prevent accidental data loss, and I designed the system to provide clear feedback to the user in such cases.

Overall, my goal with the backend was to create a solid, invisible foundation for the application, one that users don't see, but that guarantees their data is accurate, secure, and always available when needed.

4.3.3 Database Schema

To ensure robust data management for "Material Minder," I designed a logical schema that structures how information about construction projects and the materials within them is organized and stored. This schema is fundamental to the application's core functionality, allowing for efficient tracking and retrieval of project-related details.

My design is centered around three primary entities: projects, the catalog of construction items, and the specific instances of items used within each project. Each of these entities is represented by a distinct collection of records, and I've established clear links between them to accurately reflect their relationships.

First, I defined the project entity. Each record here represents an individual construction job that a user is managing. This includes a unique identifier for the project, its title, and the date associated with it. This forms the basis for organizing all material lists.

Next, I established a collection for construction items. This serves as a comprehensive catalog of all available building materials that can be referenced. Each item in this catalog is given a unique identifier, a descriptive name, an estimated price, and a reference for an image, aiding in clear identification and cost estimation.

Finally, to manage the intricate relationship where a project can include many types of materials, and a single material type can be used across multiple projects, I introduced an associative entity, which I call project items. This record acts as a crucial link between my project records and my construction item records. Importantly, within each project item record, I also store the specific quantity of a given construction item that is associated with a particular project. This allows for precise material lists and helps in calculating estimated project costs.

This streamlined approach to data organization provides a clear and efficient framework for "Material Minder" to manage all necessary information, forming a stable and adaptable foundation for its features.

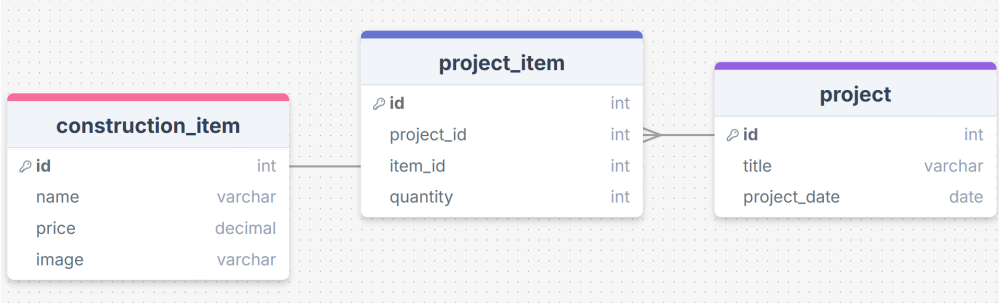

Figure 4.4: Database Schema of Material Minder.

4.4 Technologies used

With the system's architecture and data model now established, the next crucial step in developing "Material Minder" involved selecting the appropriate technologies to bring this design to life. My choices for the various components of the application, from the user interface to the underlying data management, were carefully considered to align with the core design philosophies and functional and non-functional

requirements outlined in the preceding sections. This included prioritizing ease of use, performance, maintainability, and compatibility, particularly given the specific demographic of my target users. This section will detail the key technologies I adopted for the frontend, backend, and any integrated services, explaining how each choice supports the overall vision of "Material Minder."

4.4.1 Frontend: Android - Jetpack Compose and Kotlin

For the frontend development of Material Minder, I chose to leverage the modern Android UI toolkit, Jetpack Compose, in conjunction with Kotlin. This decision was driven by the significant advantages that Jetpack Compose offers for building native Android user interfaces, accelerating and simplifying the development process with less code [Ase22].

Jetpack Compose represents a complete rewrite of the Android UI toolkit, designed from the ground up to be declarative [Ase22]. In a declarative programming paradigm, I focused on describing the desired UI state, rather than outlining every step to achieve that state, which is a departure from the imperative approach of the old Android UI system that often required more development time [Ase22]. This allowed me to define how the UI should look programmatically by describing its structure and data dependencies using composable functions [Ase22].

A key benefit I found was Jetpack Compose's approach to composition over inheritance. Unlike the older Android UI system where views inherited from parent views, Jetpack Compose uses functions to display elements on the screen. This functional approach means that to create new features, I could simply nest other composable functions within existing ones, providing a highly flexible and scalable way to buid the UI components of Material Minder [Ase22].

Furthermore, Jetpack Compose inherently supports encapsulation and a unidirectional top-down data flow [Ase22]. This design principle ensures that the UI components are a direct representation of data and are not responsible for managing state themselves. Data flows down to the UI, and events flow from the UI up, which significantly enhanced the testability and consistency of the application's user interface [Ase22]. The recomposition mechanism in Jetpack Compose was particularly useful, as it allowed UI elements to be re-invoked and updated efficiently whenever an underlying state change occurred, without the need for explicit update callbacks or LifecycleOwners [Ase22].

The choice of Kotlin as the primary language for development was also pivotal. Jetpack Compose is written in Kotlin, for Kotlin, eliminating the need to combine Java/Kotlin code with XML for UI definitions, as was common in the older system [Ase22]. This unified language approach made the codebase more concise, readable,

and allowed me to fully utilize Kotlin's modern language features and functional programming paradigms throughout the development of Material Minder [Ase22].

I utilized Android Studio as my integrated development environment (IDE) for building Material Minder. Android Studio provides powerful code editing tools, a Gradle-based build system, and Compose-specific tools that simplified my workflow and facilitated the development process [Ase22]. The project structure within Android Studio, with its module-based system, allowed me to organize the application effectively, although for Material Minder, I focused on a single application module for the core development.

4.4.2 Backend: Java Spring Boot

When it came to architecting the backend for Material Minder, my primary goal was to create a system that was not only robust and reliable but also agile enough to facilitate rapid development. This led me directly to Java Spring Boot, a choice I found incredibly empowering for constructing scalable Java applications [Man16]. It felt like bringing the core intelligence to life for my mobile app.

Underneath Spring Boot, the broader Spring Framework offered a treasure trove of modular components, each designed to tackle specific aspects of enterprise software with elegance [Man16]. I personally leaned heavily on a few key areas that proved indispensable:

- **Spring Core**: This was the bedrock. The Inversion of Control (IoC) container and Dependency Injection (DI) fundamentally changed how I thought about organizing my code. By using DI, I could build components that were beautifully decoupled, meaning changes in one part of the system wouldn't ripple uncontrollably through others. This made testing a dream and kept my codebase incredibly clean and manageable.

- **Spring Data JPA**: Handling database interactions used to be a daunting task, but Spring Data JPA transformed it. It provided this brilliant abstraction layer over JPA, allowing me to define simple interfaces for my repositories, and Spring Data JPA would just *magically* implement the complex database operations for me. I barely had to write any boilerplate code for persistent data storage! Features like dynamic query generation and built-in pagination were incredibly powerful for managing all the material lists and user data within Material Minder [Man16].

- **RESTful APIs with Spring Boot's Web Capabilities**: While I wasn't explicitly configuring Spring MVC in the old-school way (thanks to Spring Boot's smart

conventions), I built the RESTful APIs that serve as the communication backbone between my Android frontend and the backend services. This seamless interaction was crucial for real-time handling of user requests and ensuring a smooth experience for the app's users.

Ultimately, the Spring Framework's lightweight and highly adaptable nature played a huge role in ensuring Material Minder's backend was not just functional, but also maintainable and scalable. By embracing best practices, such as extending specific repository interfaces like 'JpaRepository' for its rich feature set over 'CrudRepository' [Man16], I felt confident that my data access layer was not just working, but thriving. Spring Boot truly provided an incredibly fertile ground for developing the powerful backend services that make Material Minder an effective tool for construction material management.

4.4.3 Database Architecture: Dual-Tiered Data Management

The data management strategy for Material Minder was implemented with a dual-tiered approach, meticulously separating local user-specific data from the centralized, global catalog of construction items. This architecture was designed to optimize performance, ensure data integrity, and provide a seamless user experience while preventing potential conflicts between individual user modifications and core application data.

Frontend Database: Local Room Persistence Layer

On the frontend, specifically within the Android application, I deployed a local database solution utilizing the Room Persistence Library. This decision was predicated on Room's design as an abstraction layer over SQLite, specifically engineered to simplify database access in Android applications while enhancing compile-time query validation [MY22].

The primary rationale for employing a local Room database was to provide each user with an isolated and responsive environment for managing their projects. This prevents individual user operations, such as adding, modifying, or deleting project-specific materials, from directly interfering with a shared global database. Room allowed me to define entities representing local data structures, such as individual project details and their associated materials, and Data Access Objects (DAOs) to interact with this data through intuitive APIs [MY22]. This approach ensures that user-generated data remains private and persists locally, even without a constant network connection.

```kotlin
@Dao
interface ConstructionItemDAO {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(items: List<ConstructionItem>)

    @Query("SELECT * FROM construction_item")
    suspend fun getAll(): List<ConstructionItem>

    @Query("SELECT * FROM construction_item WHERE id = :id")
    suspend fun getConstructionItemById(id: Int): ConstructionItem

    @Query("DELETE FROM construction_item WHERE id = :id")
    suspend fun deleteById(id: Int)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun updateItem(item: ConstructionItem)

    @Query("SELECT * FROM construction_item WHERE name LIKE '%' || :query || '%'")
    suspend fun searchByName(query: String): List<ConstructionItem>

    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insertOneAndGetId(item: ConstructionItem): Long
}
```

Figure 4.5: Construction Item DAO.

While direct SQLite interaction can offer marginally faster query execution in some scenarios due to bypassing an abstraction layer, Room's comprehensive benefits, including compile-time safety and a more structured approach to database management, were deemed critical for development efficiency and long-term maintainability [MY22]. The integration of Room with Android's Architecture Components, particularly ViewModel and LiveData, also streamlined the process of reflecting database changes in the UI, adhering to modern Android development best practices.

Backend Database: PostgreSQL for Centralized Data Management

For the backend, hosting the global catalog of construction items and supporting the application's core logic, I selected PostgreSQL. PostgreSQL is a powerful, open-source object-relational database system known for its robust feature set, reliability, performance, and extensibility.

PostgreSQL serves as the central repository for all standardized construction materials, including their names, estimated prices, and image references. This centralized database ensures consistency across all users of Material Minder regarding the foundational material data. The backend services, developed with Java Spring Boot, interact with PostgreSQL to retrieve global item information, calculate estimated project costs based on the latest catalogue prices, and potentially update the global catalogue by authorized administrators.

The choice of PostgreSQL aligns with the need for a scalable and reliable relational database capable of handling concurrent requests from multiple users while maintaining data integrity. Its ACID (Atomicity, Consistency, Isolation, Durability) compliance and advanced features, such as support for complex queries and transactional capabilities, provide a solid foundation for the backend's data persistence layer. This clear separation of concerns, with a local Room database for user-specific project data and a centralized PostgreSQL database for the global material catalog, optimizes both client-side performance and server-side data management.

4.4.4 Web Scraping: UiPath Studio for Data Acquisition

To populate the comprehensive global catalog of construction items for Material Minder, a significant challenge involved acquiring accurate and up-to-date product information, including names, prices, and image references, from various large material providers. Manually collecting this extensive dataset would have been an impractical and time-consuming endeavor. Consequently, I leveraged web scraping as a crucial method for efficient data acquisition.

For this purpose, I selected UiPath Studio, a leading Robotic Process Automation (RPA) tool. My decision to utilize UiPath Studio was based on its robust capabilities for automating web interactions and its intuitive visual development environment, which allowed me to design complex scraping workflows without extensive coding. Furthermore, my existing proficiency with UiPath Studio, cultivated through university coursework, significantly contributed to its selection, streamlining the development and implementation process. This was particularly beneficial given the dynamic and often intricate structures of modern e-commerce websites like Dedeman, one of the primary data sources.

The process involved designing automated workflows within UiPath Studio to systematically navigate the websites of material providers. I developed sequences that mimicked human interaction: initially, by scraping all of the website links corresponding to different product categories, as direct access to all construction items on a single page was not feasible. After successfully acquiring these category-specific links, the program then proceeded to iterate through each link, meticulously scraping every individual item. Crucially, I configured UiPath to extract structured data, specifically targeting product names, their corresponding prices, and URLs for associated images. Error handling mechanisms were also incorporated into the workflows to manage common issues such as network interruptions, unexpected pop-ups, or changes in website layout, ensuring the resilience of the scraping process.

The data extracted through these UiPath Studio workflows was then processed and standardized before being ingested into the PostgreSQL database that forms the

global material catalog. This automated web scraping approach significantly expedited the initial population of the database, providing Material Minder with a rich and relevant dataset of construction materials, which would have been unfeasible to gather through manual means.

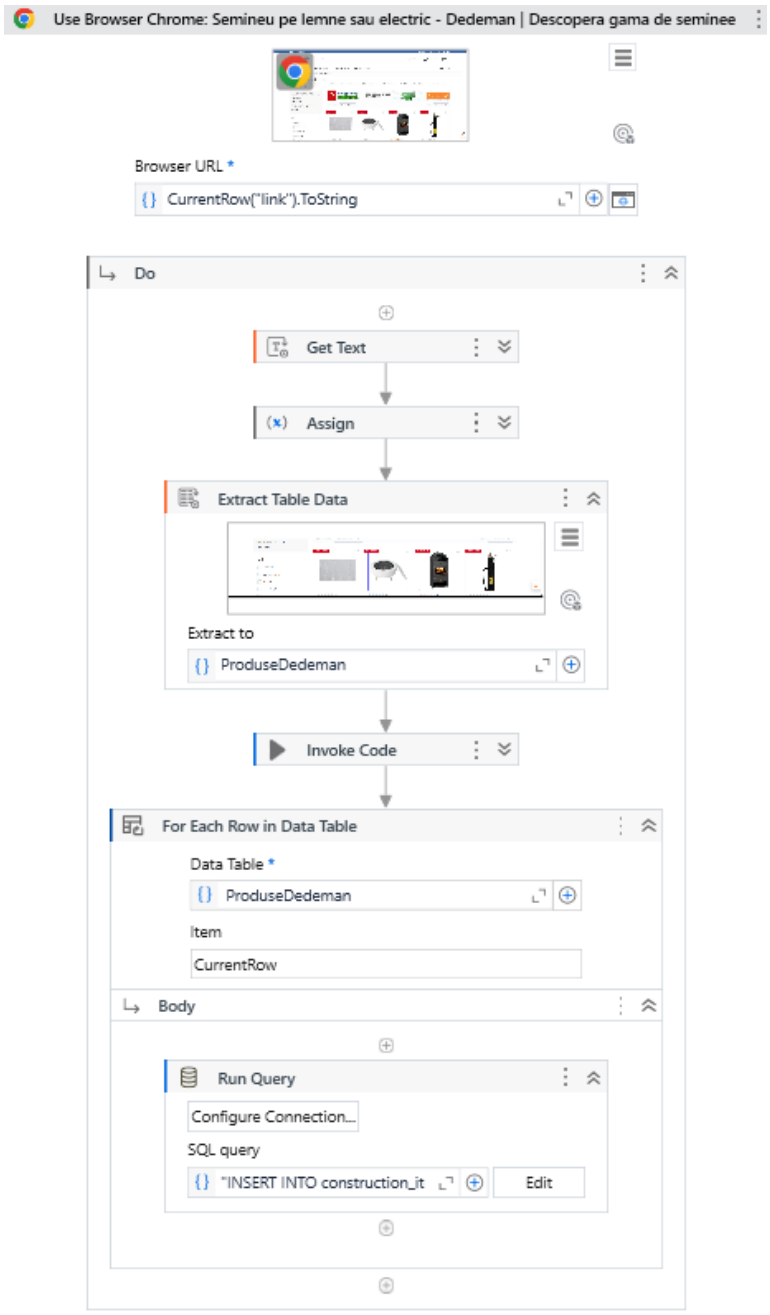

Figure 4.6: The code block which extracts the fields for the construction items.

4.5 Features and implementation details

This section delves into the core functionalities of Material Minder, providing an in-depth examination of each key feature and the underlying technical implementation. I will detail how the theoretical concepts and chosen technologies, discussed in the preceding chapters, were practically applied to develop a robust and user-friendly mobile application. From the intuitive material list creation to the integration of voice-enabled commands and seamless data management, this section outlines the architectural decisions and coding paradigms employed to bring Material Minder to fruition.

4.5.1 CRUD Features

The core functionality of Material Minder revolves around the fundamental Create, Read, Update, and Delete (CRUD) operations for managing construction projects and their associated materials. These features form the backbone of the application, enabling users to effectively organize and maintain their material lists. I meticulously designed and implemented these operations to be intuitive for the target demographic, particularly elderly users and tradespeople, adhering to the user experience principles outlined in Chapter 2.

Project Management (Create, Read, Update, Delete Projects)

- **Creating Projects**: Users are empowered to initiate new construction projects by providing a project title and an associated date. This operation captures essential metadata for organizing material lists. On the frontend, this involves user input through a clean UI, which then triggers a call to the backend. The backend service processes this request, validating the input and persisting the new project record in the PostgreSQL database.
- **Reading Projects**: The application provides a comprehensive overview of all existing projects, displayed in an easily browsable format. Users can quickly access project details, including the associated materials and estimated costs. This involves fetching project data from the backend database via optimized queries, which is then rendered on the Android frontend using Jetpack Compose components.
- **Updating Projects**: Flexibility is key for managing ongoing construction work. Users can modify existing project details, such as changing the project title or date. This process ensures that project information remains current. The frontend sends update requests to the backend, which applies the changes to the

corresponding project record in the PostgreSQL database, maintaining data integrity.

- **Deleting Projects**: Users have the ability to remove projects that are no longer needed. This operation includes appropriate confirmation steps to prevent accidental data loss. When a project is deleted, the backend ensures that all associated materials within that project are also removed from the database, maintaining data consistency and cleanliness.

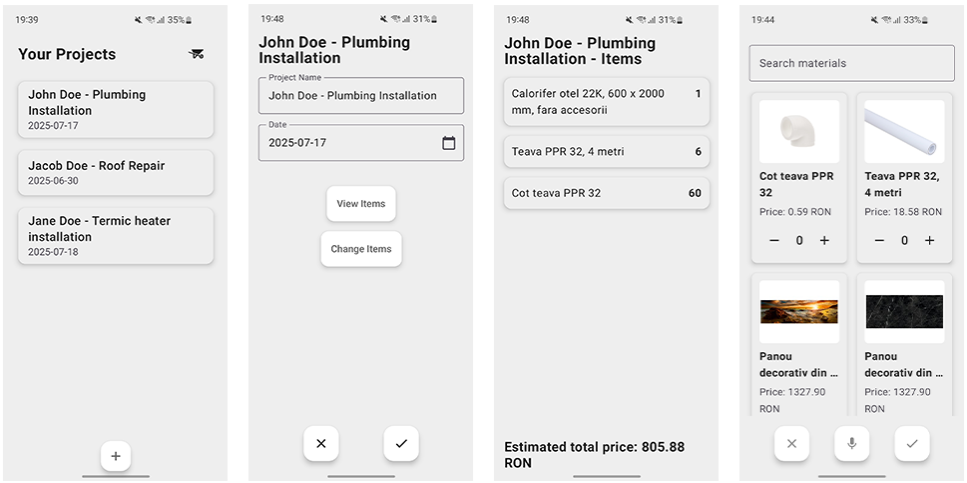

Figure 4.7: Screenshots of the project management screens from the UI.

Material Management (Create, Read, Update, Delete Materials within Projects)

- **Adding Materials to Projects (Create)**: This is a critical and frequently used feature. Users can add specific construction items to an active project. While the core "add" functionality is facilitated by Speech-to-Text (detailed in a subsequent section), the underlying CRUD mechanism involves associating an item from the global catalog (or a user-defined local item) with a project and specifying its quantity. This data is stored locally in the Room database, linking to the project ID and material ID.

- **Viewing Materials within Projects (Read)**: For each selected project, the application displays a simplified list of its associated construction items, showing the item name and its quantity. This allows for quick verification of required materials. The frontend queries the local Room database to retrieve this project-specific material list, ensuring rapid access and display.

- **Updating Material Quantities (Update)**: Users can adjust the quantity of any material already added to a project. This flexibility is essential for adapting

to real-world construction needs. The update operation targets the specific material entry in the local Room database for the given project, modifying only the quantity attribute.

- **Deleting Materials from Projects (Delete)**: Individual materials can be removed from a project list. This operation is designed to be straightforward, allowing users to easily correct mistakes or adjust requirements. The corresponding entry is removed from the local Room database for that particular project.

All CRUD operations are designed with robust error handling and immediate user feedback mechanisms to enhance usability and reliability, ensuring that users are always aware of the system's state following their interactions.

4.5.2 Speech-to-Text Integration for Construction Item List Creation

Core Speech Recognition Setup:

At the heart of this feature is Android's built-in SpeechRecognizer class. To manage its lifecycle and operations cleanly, I encapsulated the speech recognition logic within a SpeechToTextService. This service is responsible for initializing the SpeechRecognizer, starting, and stopping the listening process.

```
class SpeechToTextService : Service() {

    private val binder = SpeechToTextBinder()
    private var speechRecognizer: SpeechRecognizer? = null
    private var recognitionListener: RecognitionListener? = null

    override fun onBind(intent: Intent?): IBinder {
        return binder
    }

    override fun onCreate() {
        super.onCreate()
        speechRecognizer = SpeechRecognizer.createSpeechRecognizer(this)
    }

    override fun onDestroy() {
        super.onDestroy()
        speechRecognizer?.destroy()
    }

    fun startListening(listener: RecognitionListener) {
        recognitionListener = listener
        val intent = Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
            putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.LANGUAGE_MODEL_FREE_FORM)
            putExtra(RecognizerIntent.EXTRA_LANGUAGE, "ro-RO")
        }
        speechRecognizer?.setRecognitionListener(listener)
        speechRecognizer?.startListening(intent)
    }

    fun stopListening() {
        speechRecognizer?.stopListening()
    }

    inner class SpeechToTextBinder : Binder() {
        fun getService(): SpeechToTextService = this@SpeechToTextService
    }
}
```

Figure 4.8: The implementation code of the SpeechToTextService.

I implemented a ServiceConnection to bind my UI (the SelectConstructionItemsScreen and ChangeConstructionItemsScreen composables) to this SpeechToTextService. This binding allows the screen to control the STT service and receive results. The isBound state variable tracks the connection status, ensuring that I only interact with the service when it's properly connected.

Handling User Interaction and Permissions:

The user initiates speech input by tapping a microphone icon (FloatingActionButton with Icons.Filled.Mic). When this button is tapped:

If not currently recording, I set an isRecording state to true and call startRecordingInternal(). If already recording, I call speechService?.stopListening() to halt the recognition process and set isRecording to false. Before starting the recording, startRecordingInternal() checks for the RECORD AUDIO permission. If the permission isn't granted, I use rememberLauncherForActivityResult with ActivityResultContracts.RequestPermission() to request it from the user. Only upon granting the permission does the actual speechService?.startListening(recognitionListener) call proceed. If the service isn't bound or ready, a Toast message informs the user.

Capturing and Processing Speech:

The SpeechToTextService uses a RecognitionListener to get feedback and results from the SpeechRecognizer. I defined a custom recognitionListener object within my composable screen. Key callback methods I focused on are:

onReadyForSpeech(): Indicates the recognizer is ready. onError(): Handles various speech recognition errors (e.g., no match, network issues, permission denial), displaying appropriate Toast messages to the user and resetting the isRecording state. onEndOfSpeech(): Called when the user stops speaking. onResults(): This is the crucial callback where the recognized speech is delivered. The results arrive as a Bundle, from which I extract a list of possible text transcriptions using SpeechRecognizer.RESULTS RECOGNITION. I typically join these into a single string.

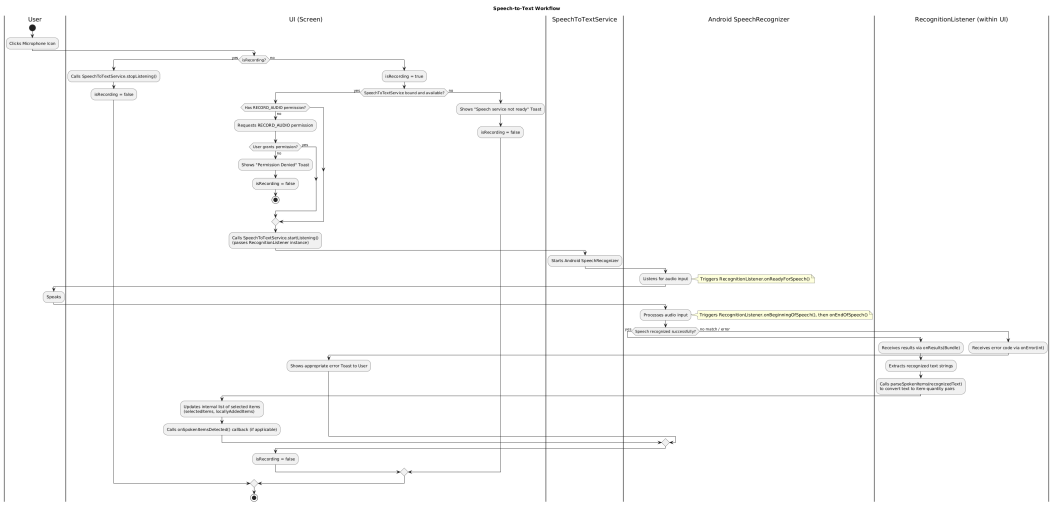

Figure 4.9: Workflow diagram of the Speech-To-Text feature.

Parsing Spoken Text into Items and Quantities:

Once I have the raw recognized text, the next step is to parse it into meaningful item names and their corresponding quantities. For this, I created a helper function, parseSpokenItems (and a similar parseSpokenItemsChange for a slightly different context).

This function employs regular expressions (Regex) to identify patterns in the spoken text. The primary pattern I look for is

```
(.*?)\s+([a-zA-ZăâîșțĂÂÎȘȚ]+)\s+bucăți\b
```

This regex aims to capture: * Group 1: (.*?) - The item name (non-greedy match for any characters). * Group 2: ([a-za-ZăâîșțĂÂÎȘȚ]+) - The quantity, expressed as a Romanian word. * `\s+bucăți\b`: The Romanian word for "pieces" or "units", acting as a delimiter. If this pattern doesn't match, I try a simpler regex

```
(.*?)\\s+bucăți\\b
```

to capture just an item name assuming a quantity of 1, or as a last resort, take the whole input as an item name with quantity 1.

To convert spoken Romanian numbers (like "doi", "cinci", "unsprezece") into integers, I implemented another helper function, convertRomanianNumberWordToInt. This function uses a when statement to map normalized (lowercase, diacritics removed) number words to their integer equivalents. It also attempts toIntOrNull() for cases where the user might say a digit directly.

Updating the UI with Parsed Items:

After parseSpokenItems returns a list of Pair¡String, Int¿ (item name, quantity), the onResults callback in recognitionListener processes these pairs:

I maintain a selectedItems SnapshotStateMap which stores the items and their quantities. The key for this map can be an item's ID (if it's an existing item from constructionObjectsList) or its name (if it's a newly spoken item not yet in the database). I also have a locallyAddedItems mutableStateListOf¡ConstructionItem¿ to keep track of items spoken by the user that don't exist in the constructionObjectsList yet. These are temporary until confirmed. For each parsed (spokenName, spokenQuantity): 1. I first check if an item with spokenName (case-insensitive) exists in the constructionObjectsList (items from the database). If yes, I update its quantity in newSelectedItemsMap using its ID as the key. 2. If not found there, I check if it exists in the currentLocallyAddedList (a temporary mutable copy of locallyAddedItems). If yes, I update its quantity in newSelectedItemsMap using its name as the key. 3. If it's a completely new item, I create a new ConstructionItem object (with a null ID and default price) and add it to currentLocallyAddedList. Then, I add it to newSelectedItemsMap with its name as the key and the spokenQuantity.

After processing all spoken items, if any changes were made (madeChanges = true), I update the main selectedItems map and locallyAddedItems list with the new state. This, due to Jetpack Compose's reactive nature, automatically triggers a recomposition, and the UI reflects the newly added or updated items. The onSpokenItemsDetected callback is also invoked, potentially for further actions in the ViewModel.

Lifecycle Management:

To prevent resource leaks, I use LaunchedEffect(Unit) to bind to the SpeechToTextService when the composable enters the composition and DisposableEffect(Unit) to unbind from the service (calling context.unbindService(serviceConnection)) and stop any active listening when the composable leaves the composition. This ensures the SpeechRecognizer is properly released.

This implementation provides a hands-free way for users to populate their construction item lists, leveraging Android's native speech recognition capabilities within a reactive Jetpack Compose UI. The parsing logic is tailored for Romanian number words and the specific phrase "bucăți" to identify quantities.

4.6 Testing and Evaluation

This section outlines the comprehensive testing and evaluation methodologies employed to assess the functionality, performance, and usability of Material Minder. Throughout the development lifecycle, I adopted a systematic approach to ensure that the application met its design objectives, adhered to specified requirements, and delivered a seamless experience for its target users. This chapter details the various testing phases conducted, including manual testing, automated acceptance testing, and detailed unit and integration testing, along with the metrics and criteria used to evaluate the application's overall success and identify areas for future improvement.

Manual Testing

Complementing the automated testing procedures, a significant portion of the evaluation process for Material Minder involved comprehensive manual testing. This approach was critical for identifying issues that automated scripts might overlook, particularly those related to user experience, visual aesthetics, and intuitive interaction flow. My primary objective with manual testing was to simulate real-world usage scenarios and assess the application's responsiveness, stability, and overall usability from a human perspective.

I systematically conducted manual tests across various facets of the application. This included rigorous checks of the user interface on different Android device emulators to ensure consistent rendering and responsiveness across various screen sizes and resolutions. Each core feature, from the CRUD operations for project and material management to the nuances of the Speech-to-Text integration for item creation, was meticulously tested. This involved entering diverse voice commands, testing edge cases for material quantities, and navigating through all possible user flows to uncover any unexpected behavior or logical discrepancies.

Furthermore, manual testing was instrumental in evaluating the overall user journey, focusing on aspects such as ease of navigation, clarity of feedback messages, and the intuitive placement of UI elements, especially considering the thesis's focus on elderly users. Any identified bugs, inconsistencies, or areas of friction in the user experience were meticulously documented, prioritized, and subsequently addressed, ensuring a polished and user-friendly final product. This iterative process of manual testing and refinement was vital in enhancing the application's quality beyond what automated checks alone could achieve.

Automated Acceptance Testing with Cucumber

To bridge the gap between business requirements and technical implementation, and to ensure that Material Minder consistently met user expectations from a behavioural perspective, I incorporated automated acceptance testing using Cucumber. Cucumber, as a Behaviour-Driven Development (BDD) framework, allowed me to define application features and their expected behaviours in a clear, human-readable format using Gherkin syntax (Given-When-Then).

I developed feature files that described various user stories, such as "adding a material to a project" or "updating a material quantity using voice commands." Each scenario within these feature files represented a specific user interaction, written in plain language that was understandable by non-technical stakeholders. These scenarios were then linked to step definitions which contained the actual code to interact with the application and verify its behaviour. This approach ensured that tests were directly tied to user requirements and served as living documentation of the system's intended functionality.

Automated acceptance tests with Cucumber were crucial for validating end-to-end user flows and ensuring that the integrated system behaved as expected from a user's perspective. They provided a high-level safety net, quickly highlighting any deviations from the desired functionality and thereby reinforcing the application's alignment with its core design principles and user needs.

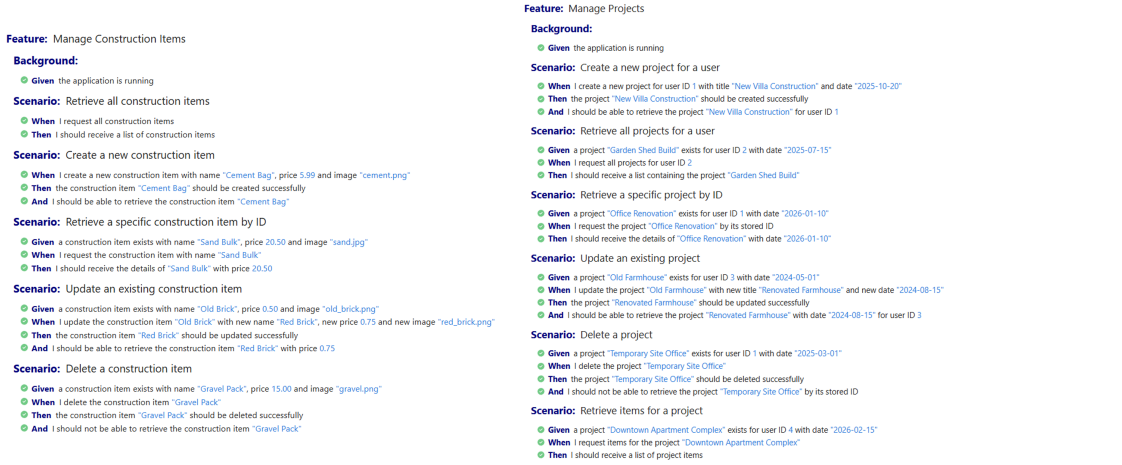

Figure 4.10: Cucumber Tests Report.

Unit and Integration Testing with JUnit and Mockito

To ensure the robustness and reliability of Material Minder's codebase at a granular level, I implemented a comprehensive strategy encompassing both unit and inte-

gration testing. These automated testing methodologies were crucial for verifying the correctness of individual components and their interactions, complementing the broader scope of manual testing and higher-level acceptance tests.

Unit Testing For unit testing, I utilized JUnit, the widely adopted testing framework for Java and Kotlin. The primary objective of unit tests was to isolate and validate the smallest testable parts of the application, such as individual functions, methods, or classes, in isolation from their dependencies. I developed numerous unit tests for both the Android frontend (Kotlin code within composables, view models, and utility functions) and the Java Spring Boot backend (service layers, utility classes, and business logic components). These tests allowed me to quickly verify the correctness of logic, handle edge cases, and ensure that refactoring efforts did not introduce regressions, contributing significantly to code quality and maintainability.

Integration Testing Building upon the foundation of unit tests, integration testing was performed to verify the interactions between different modules or services within Material Minder. While unit tests focus on isolated components, integration tests examine how these components work together. For instance, on the backend, I tested the interaction between the service layer and the data access layer (repositories) to ensure proper data persistence and retrieval. Similarly, on the frontend, I validated how view models correctly interacted with Room database operations. For these tests, Mockito was an invaluable tool. Mockito allowed me to create mock objects for external dependencies (such as database repositories, external APIs, or complex service components) that were not the direct subject of the integration test. By mocking these dependencies, I could control their behavior and ensure that the focus remained solely on the integration logic being tested, preventing external factors from influencing test outcomes and making tests more predictable and reliable.

4.7 Future of Material Minder

The current iteration of Material Minder establishes a solid foundation for voice-enabled construction material management. However, the rapidly evolving landscape of mobile technology and artificial intelligence presents numerous opportunities for significant expansion and enhancement. This section outlines the envisioned future trajectory for Material Minder, detailing key functionalities and strategic directions that would further elevate its utility and user experience.

4.7.1 Cross-Platform Accessibility and Synchronization

A paramount future development for Material Minder involves extending its availability beyond the Android platform to encompass a truly cross-platform ecosystem. This includes developing native applications for iOS, as well as web-based and desktop versions. The goal is to enable users to seamlessly access and manage their construction projects and material lists from any device, be it a smartphone, tablet, laptop, or desktop computer, ensuring continuous access and synchronized data across all touchpoints. Implementing this would involve exploring frameworks that facilitate multi-platform development, minimizing code duplication while maintaining native performance and user experience where appropriate. This expansion would significantly enhance user convenience and adaptability to diverse work environments.

4.7.2 AI-Powered Material Suggestion and Optimization

An exciting future enhancement is the integration of an intelligent AI model designed to provide smart material suggestions during the list creation process. This AI would analyze historical project data, common construction practices, and potentially even user-specific patterns to proactively suggest relevant materials or optimize quantities. For instance, based on a user verbally listing "concrete mix," the AI might suggest related items like "rebar," "shovels," or "waterproofing agents," alongside estimated quantities. This feature would not only streamline the list-building process but also reduce oversight, enhance accuracy, and potentially suggest more cost-effective alternatives, transforming Material Minder into a more proactive and intelligent assistant.

4.7.3 Advanced Speech-to-Text (STT) Capabilities

While the current Speech-to-Text feature is functional for basic item addition, future efforts would focus on significant enhancements to its robustness and intelligence. This includes improving transcription accuracy in challenging environments, such as construction sites with ambient noise, and developing a more sophisticated understanding of construction-specific jargon and regional dialects. Furthermore, advancements could involve integrating more complex voice commands for actions beyond simple addition (e.g., "increase quantity of bricks by ten," "move cement to finished items"). Exploring features like speaker diarization could also enable the application to differentiate between multiple voices, facilitating collaborative list creation scenarios.

4.7.4 Collaborative Project Management

Given that construction projects often involve multiple stakeholders, implementing collaborative features would be a logical and highly beneficial extension. This could include functionalities such as sharing project lists with team members, allowing real-time updates and concurrent modifications by authorized users, and providing role-based access controls. Such features would foster better communication, reduce errors, and enhance overall team efficiency on multi-person construction sites.

4.7.5 Analytics and Reporting Features

Implementing basic analytics and reporting tools within the application would offer users valuable insights into their material consumption patterns and project expenditures over time. Features like cost breakdown reports, material usage trends, and budget tracking could assist in better planning, cost control, and identifying areas for efficiency improvements in future projects.

These prospective developments aim to evolve Material Minder from a practical utility into a comprehensive, intelligent, and interconnected platform that supports the modern construction professional across all facets of material management.

Chapter 5

Conclusion

This thesis presented the design, development, and evaluation of Material Minder: a voice-enabled mobile application engineered to streamline the management of construction materials, particularly addressing the needs of tradespeople, including elderly users, through an intuitive and accessible interface. The project successfully tackled the challenge of inefficient material list creation by integrating modern mobile development practices with advanced speech-to-text capabilities.

Throughout this endeavor, I employed a multi-faceted approach to system development. The frontend, built on Android using Jetpack Compose and Kotlin, emphasized a declarative UI design, ensuring responsiveness and a user-friendly experience. For the backend, Java Spring Boot provided a robust and scalable framework for handling business logic and data persistence, supported by a PostgreSQL database for the centralized material catalog. A dual-tiered database architecture was implemented, utilizing a local Room database on the frontend for user-specific project data and PostgreSQL on the backend for the global item repository, optimizing both performance and data integrity. Furthermore, I leveraged UiPath Studio for automated web scraping, which proved instrumental in efficiently populating the extensive global material catalog from various providers.

Material Minder stands as a testament to the potential of leveraging contemporary software engineering paradigms and voice technology to solve real-world problems in the construction sector. It simplifies a critical workflow, empowering users with an efficient and accessible tool for material management. While the current version achieves its primary objectives, the outlined future developments promise to further enhance its capabilities, extending its reach and intelligence to become an even more indispensable companion for construction professionals.

Bibliography

[AC20] Inthraporn Aranyanak and Pattama Charoenporn. Ux-based design of a mobile application for thai seniors. In *Proceedings of the 6th International Conference on Frontiers of Educational Technologies*, pages 160–163, 2020.

[Ase22] Beselam Gizachew Asefa. Building android component library using jetpack compose. 2022.

[BAJ+23] Sneha Basak, Himanshi Agrawal, Shreya Jena, Shilpa Gite, Mrinal Bachute, Biswajeet Pradhan, and Mazen Assiri. Challenges and limitations in speech recognition technology: a critical review of speech signal processing algorithms, tools and systems. *CMES-Computer Modeling in Engineering and Sciences*, 2023.

[CW16] Eric Corbett and Astrid Weber. What can i say? addressing user experience challenges of a mobile voice user interface for accessibility. In *Proceedings of the 18th international conference on human-computer interaction with mobile devices and services*, pages 72–82, 2016.

[Dat25] Datex Corporation. How voice-picking is optimizing warehousing operations, 2025. Accessed on: June 4, 2025.

[DBM14] José-Manuel Díaz-Bossini and Lourdes Moreno. Accessibility to mobile interfaces for older people. *Procedia computer science*, 27:57–66, 2014.

[Dob22] Alexandra Dobre. Decalajul digital și excluziunea digitală a vârstnicilor în românia - un studiu de caz în București-ilfov. *Calitatea Vieții*, 33(4):264–284, Dec. 2022.

[Gar03] Jesse James Garrett. *The Elements of User Experience: User-Centered Design for the Web*. New Riders Press, Berkeley, CA, USA, 2003. Archived 3 December 2016 at the Wayback Machine. Accessed via the author's official website.

[HGK18] Shravankumar Hiregoudar, Manjunath Gonal, and KG Karibasappa. Speech to sql generator-a voice based approach. *Journal of Basic and Applied Research International*, 4:01–05, 2018.

[JG07] Hans-Christian Jetter and Jens Gerken. A simplified model of user experience for practical application. 2007.

[Man16] Ramakrishna Manchana. Building scalable java applications: An in-depth exploration of spring framework and its ecosystem. *International Journal of Science Engineering and Technology*, 4:1–9, 2016.

[MY22] Abdul-Rahman Mawlood-Yunis. Android sqlite, firebase, and room databases. In *Android for Java Programmers*, pages 475–530. Springer, 2022.

[N+15] Radka Nacheva et al. Principles of user interface design: important rules that every designer should follow. *Izvestia Journal of the Union of Scientists-Varna. Economic Sciences Series*, 1:140–149, 2015.

[SBT+22] Mudita Sandesara, Umesh Bodkhe, Sudeep Tanwar, Mohammad Dahman Alshehri, Ravi Sharma, Bogdan-Constantin Neagu, Gheorghe Grigoras, and Maria Simona Raboaca. Design and experience of mobile applications: a pilot survey. *Mathematics*, 10(14):2380, 2022.

[SK25] We Are Social and Kepios. Digital 2025 Romania. `https://datareportal.com/reports/digital-2025-romania`, 2025. Accessed on: June 2, 2025.

[TAHPC24] Bruno Tiglla-Arrascue, Junior Huerta-Pahuacho, and Luis Canaval. Speech recognition for inventory management in small businesses. *ICSBT 2024*, page 95, 2024.

[TPS+18] Ayushi Trivedi, Navya Pant, Pinal Shah, Simran Sonik, and Supriya Agrawal. Speech to text and text to speech recognition systems-areview. *IOSR J. Comput. Eng*, 20(2):36–43, 2018.

[Yah24] Yahoo News. Buymeacoffee's founder built an ai-powered productivity tool called voicenotes, 2024. Accessed on: June 5, 2025.

[YLL16] You-Dong Yun, Chanhee Lee, and Heui-Seok Lim. Designing an intelligent ui/ux system based on the cognitive response for smart senior. In *2016 2nd International Conference on Science in Information Technology (ICSITech)*, pages 281–284, Balikpapan, Indonesia, 2016. IEEE.

[YR19] Shunguo Yan and PG Ramachandran. The current status of accessibility in mobile apps. *ACM Transactions on Accessible Computing (TACCESS)*, 12(1):1–31, 2019.